

Rapport TP IHM

Projet G4

Réalisé par :

Anis TEZKRATT

Islem MEZIOUD

Aris SAHRAOUI

Introduction

Ce projet consiste en le développement d'une application bureautique de gestion de contacts personnels et professionnels (carnet d'adresses).

Le projet est publié sur GitHub sous des licences ouvertes : le code source est sous licence MIT, et les ressources de conception (maquettes Figma, diagrammes) sont sous licence Creative Commons Attribution 4.0 International (CC BY 4.0).

Problématique

Comment concevoir et développer une application desktop locale, structurée selon le modèle MVC, permettant une gestion complète des contacts via des opérations CRUD ?

Objectif

Développer une application bureautique locale permettant la gestion complète de contacts personnels et professionnels via une interface graphique intuitive.

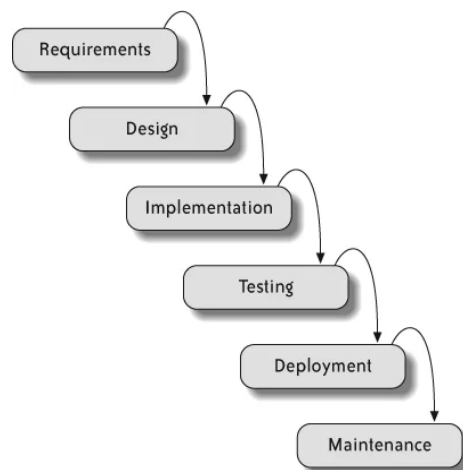
Technologies utilisées

Le projet repose sur **Python** comme langage principal, avec **Tkinter** pour l'interface graphique et **ttkbootstrap** pour l'amélioration visuelle (thèmes dark/light). La persistance des données est assurée par **SQLite** via le module `sqlite3`, et le code est structuré selon l'architecture **MVC**. Les outils de développement utilisés incluent VS Code comme éditeur, GitHub pour l'hébergement et le versionnement du code source, **Google Docs** pour la rédaction collaborative, **Figma** et **Draw.io** pour la conception des maquettes et diagrammes, et **PlantUML** pour la modélisation UML.

SQLiteStudio et DB Browser for SQLite pour la visualisation et l'inspection de la base de données.

Justification du choix de la méthodologie Waterfall

La méthodologie Waterfall (par cascade) est adaptée ici car les exigences étaient claires, stables et définies dès le départ. L'équipe peut donc suivre un déroulement séquentiel des phases :



Ce cadre méthodologique favorise :

- Une documentation structurée (requirements/design) et une implémentation fidèle au cahier des charges, finalement une validation progressive (revue et tests en fin de phase).

1- Analyse des besoins (Requirements)

Profil Utilisateur

L'application s'adresse à toute personne en grand public souhaitant gérer ses contacts de façon simple et organisée, dans un contexte personnel ou professionnel.

Caractéristiques :

Niveau technique : de débutant à intermédiaire.

Besoins principaux : ajouter, rechercher, modifier, supprimer et consulter des contacts.

Exigences : interface intuitive, rapidité, fiabilité des données, pas de compétences en base de données requises.

Scénarios types :

Ajout d'un nouveau contact après une rencontre.

Recherche rapide d'un numéro ou d'un email.

Mise à jour ou suppression d'un contact existant.

Description des cas d'utilisation

Textuellement:

1. Modélisation des tâches (cas d'utilisation)

Acteur principal : Utilisateur

Tâche 1 : Ajouter un contact

Flux principal :

L'utilisateur saisit les informations du contact puis valide l'ajout. Le système enregistre le contact dans la base de données et met à jour automatiquement la liste des contacts.

Flux alternatifs :

Si les champs obligatoires ne sont pas renseignés, un message d'erreur est affiché. En cas d'erreur lors de l'enregistrement, l'opération est annulée et un message d'échec est affiché.

Tâche 2 : Rechercher un contact

Flux principal :

L'utilisateur saisit un critère de recherche (nom, téléphone ou email). Le système effectue une recherche avec correspondance partielle et affiche les contacts correspondants.

Flux alternatifs :

Si aucun contact ne correspond au critère, un message informatif est affiché.

Tâche 3 : Modifier un contact

Flux principal :

L'utilisateur sélectionne un contact dans la liste. Les informations sont chargées dans le formulaire, puis modifiées et validées. Le système met à jour les données dans la base.

Flux alternatifs :

Si aucun contact n'est sélectionné, un message d'erreur est affiché. En cas d'échec de la mise à jour, les modifications ne sont pas enregistrées.

Tâche 4 : Supprimer un contact

Flux principal :

L'utilisateur sélectionne un contact et confirme la suppression. Le système supprime le contact de la base et met à jour la liste.

Flux alternatifs :

Si l'utilisateur annule la confirmation, aucune suppression n'est effectuée.

Tâche 5 : Afficher tous les contacts

Flux principal :

Le système affiche l'ensemble des contacts enregistrés, triés alphabétiquement par nom.

Flux alternatifs :

Si aucun contact n'est enregistré, la liste affichée est vide.

Tâche 6 : Quitter l'application

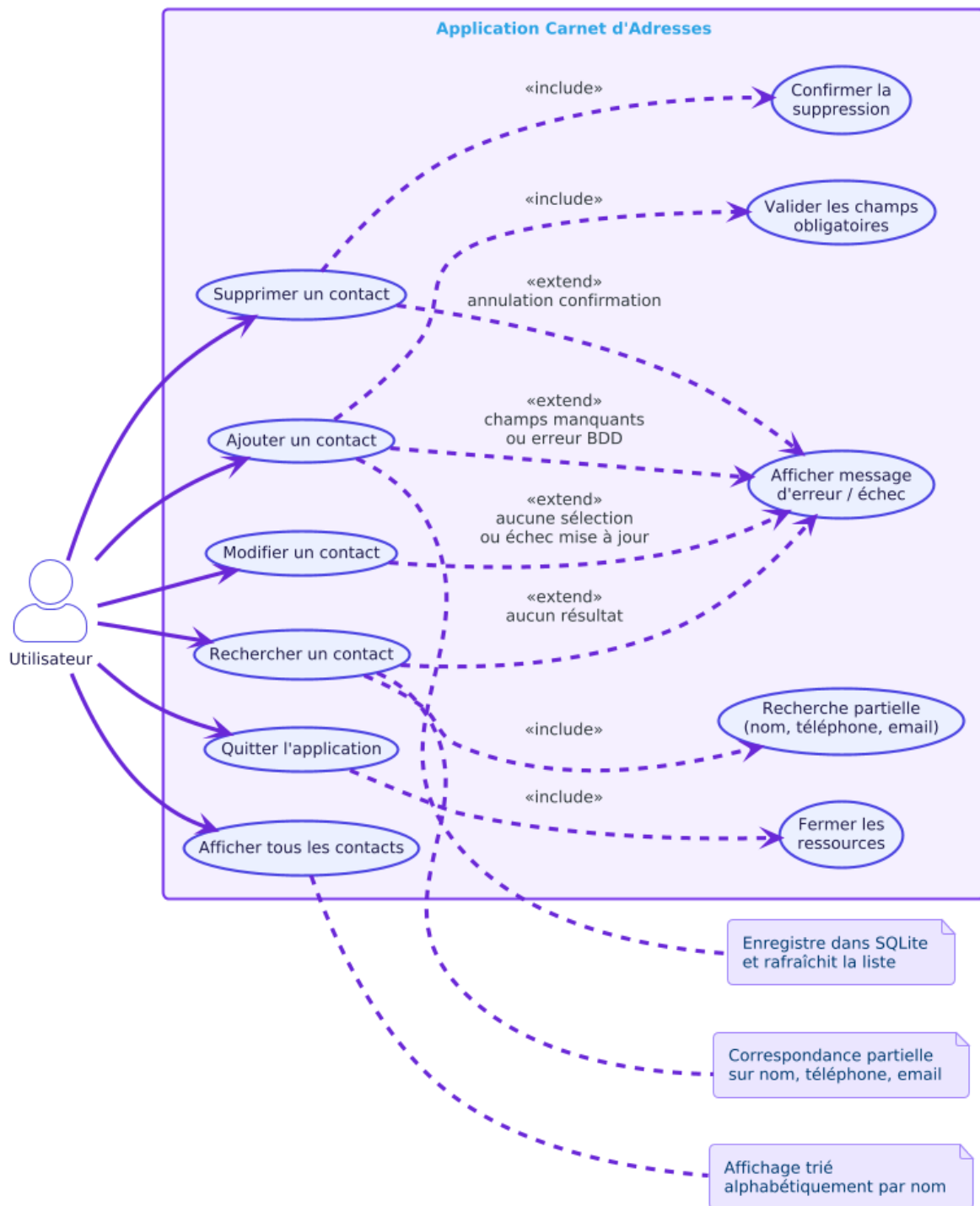
Flux principal :

L'utilisateur quitte l'application. Le système ferme correctement les ressources et termine l'exécution.

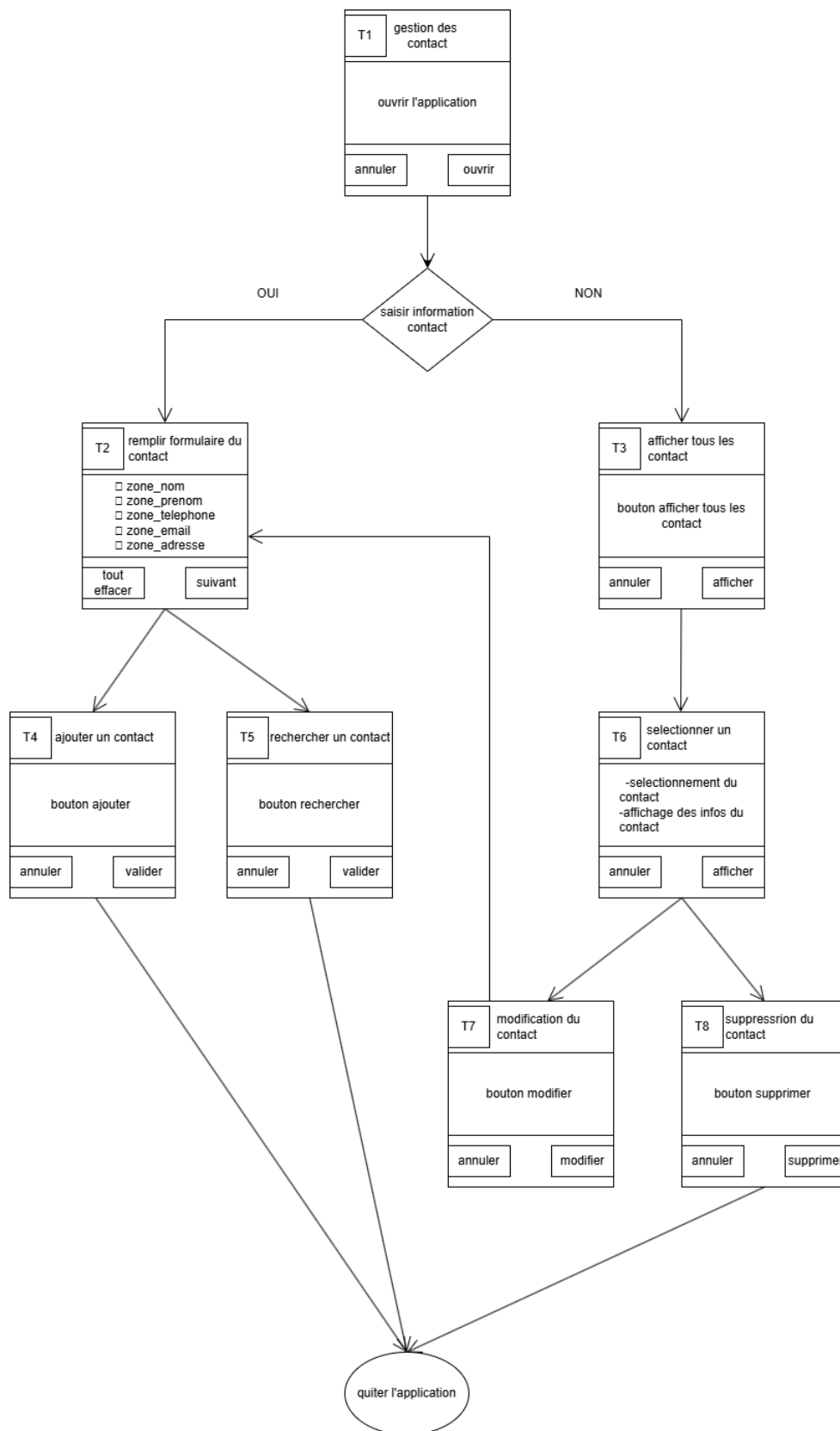
Note :

Afin de mieux visualiser et comprendre les différents flux d'interaction, il est recommandé de consulter le diagramme *User Flow* fourni sous forme de fichier .drawio.

Diagramme cas d'utilisation (UML)



IHM Abstraite



3. Conception (Design)

3.1 IHM Concrete (Design de l'interface)

Suite à l'analyse des besoins fonctionnels et ergonomiques, la conception de l'interface a été réalisée selon une approche structurée et itérative à l'aide de **Figma**. Nous avons d'abord produit une maquette statique (mockup) permettant de valider la disposition spatiale des éléments, la lisibilité des champs (Nom, Prénom, Téléphone, Email, Adresse) et la conformité aux exigences de l'énoncé (formulaire de saisie, boutons d'action, zone d'affichage tabulaire). Cette maquette a été exportée aux formats PDF et PNG pour documentation. Elle a ensuite été transformée en un prototype interactif simulant le comportement complet de l'application : saisie avec validation, ajout, modification, suppression et recherche avec feedback visuel. Le prototype est accessible via un lien public permettant de tester les parcours utilisateurs. Chaque étape a été guidée par le respect des heuristiques de Bastien & Scapin.

Un fichier au format **.fig** (éditable dans Figma) est également disponible dans le dépôt GitHub.

La maquette d'interface utilisateur pour un carnet d'adresses est présentée dans une fenêtre intitulée "Carnet d'adresses". La barre de menu comprend "Fichier", "édition", "Affichage" et "Aide".

Le formulaire "Informations du contact" est centré et contient les champs suivants :

- Nom :
- Prénom :
- Téléphone :
- Email :
- Adresse :

Un bouton "Effacer" est situé en bas du formulaire.

En dessous du formulaire, une barre de boutons contient :

- Ajouter (icône +)
- Rechercher (icône loupe)
- Modifier (icône crayon)
- Supprimer (icône poubelle)
- Afficher tous (icône document)
- Quitter (icône porte)

La section d'affichage des contacts est structurée comme suit :

Nom	Prénom	Téléphone	Email	Adresse
-----	--------	-----------	-------	---------

En bas de la fenêtre, le texte "Contacts affichés :" est visible.

L'interface a été conçue et évaluée selon les 8 critères ergonomiques de Bastien & Scapin, garantissant une expérience utilisateur intuitive et efficace. Le tableau ci-dessous résume la mise en œuvre de chaque heuristique :

Heuristique	Principe	Mise en œuvre dans l'application
1. Guidage	Aider l'utilisateur à accomplir ses tâches.	Libellés explicites, messages contextuels (contact ajouté), validation en temps réel de l'email.
2. Charge de travail	Réduire la charge cognitive et physique.	Formulaire compact, boutons d'action clairs, double clic pour pré-remplir le formulaire, affichage trié par défaut.
3. Contrôle explicite	L'utilisateur doit garder le contrôle.	Confirmation obligatoire avant suppression, annulation possible, actions déclenchées uniquement par des boutons (Ajouter, Modifier).
4. Adaptabilité	S'adapter aux besoins et habitudes de l'utilisateur.	Interface modulable (thèmes interchangeables via ttkbootstrap) et champs optionnels permettant une saisie flexible selon le contexte d'usage.
5. Gestion des erreurs	Prévenir, expliquer et aider à corriger les erreurs.	Messages d'erreur spécifiques (email invalide, champ obligatoire vide), champs non effacés après une erreur.
6. Homogénéité / Cohérence	Maintenir des conventions similaires dans toute l'interface.	Disposition constante des zones (formulaire en haut, actions au centre, liste en bas), conventions Tkinter standards, terminologie uniforme.
7. Signification des codes et dénominations	Utiliser un langage clair et familier.	Intitulés des champs et boutons sans ambiguïté (Nom, Rechercher, Afficher tous), icônes éventuelles explicites.
8. Compatibilité	Respecter les attentes et habitudes de l'utilisateur.	Métaphore du carnet d'adresses papier traduite en numérique, comportement attendu pour la recherche (correspondance partielle), tri alphabétique classique.

3.3 Architecture MVC

Le choix de l'architecture MVC (Model-View-Controller) est motivé par plusieurs impératifs de qualité logicielle. Elle assure une séparation claire des responsabilités : le Modèle gère exclusivement l'accès et la logique des données via SQLite, la Vue se consacre à l'interface graphique (Tkinter, avec une version améliorée view2.py utilisant ttkbootstrap), et le Contrôleur orchestre les interactions et valide les entrées. Cette séparation facilite les tests unitaires, en particulier sur la couche modèle, simplifie la maintenance et l'évolution (comme le remplacement ou l'enrichissement de la vue), et améliore la réutilisabilité des composants.

Structure des fichiers

L'application est organisée en cinq modules Python :

model.py : Opérations CRUD sur la base SQLite.

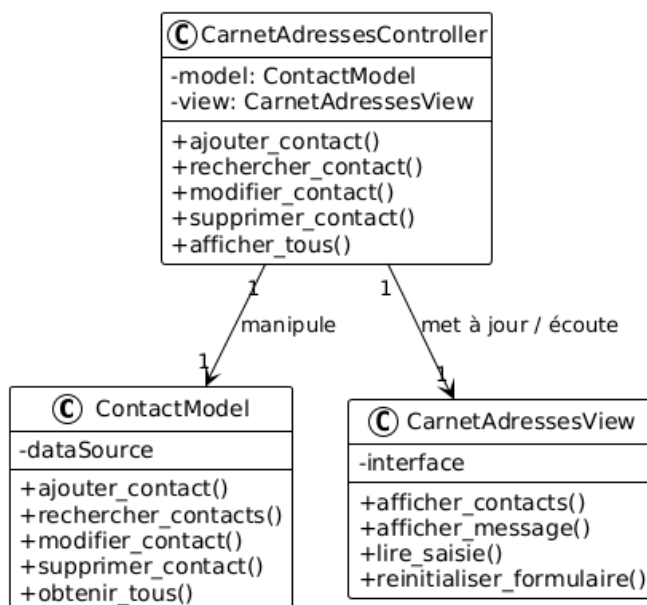
view.py : Interface Tkinter standard (formulaire, Treeview, boutons).

view2.py : Version enrichie de la vue avec ttkbootstrap pour des thèmes modernes et une meilleure expérience visuelle.

controller.py : Validation des saisies et coordination entre la vue et le modèle.

main.py : point d'entrée qui initialise les trois composants MVC et lance la boucle principale de Tkinter.

Pour illustrer clairement les relations et responsabilités entre ces composants, le diagramme de classes UML ci-dessous présente la structure interne de l'application, mettant en évidence les attributs, méthodes et dépendances de chaque classe.



4. Développement (Implementation)

4.1 Architecture et implémentation MVC

L'architecture Modèle-Vue-Contrôleur (MVC), définie lors de la phase de conception, a été rigoureusement implémentée selon le découpage spécifié dans le cahier des charges. La programmation orientée objet (POO) a été systématiquement utilisée pour structurer chaque composant, garantissant ainsi une encapsulation claire, une réutilisabilité optimale et une séparation effective des responsabilités au sein du pattern MVC.

4.2 Implémentation du modèle (model.py)

4.2.1 Connexion et initialisation de la base de données

La classe `ContactModel` utilise le module `sqlite3` pour créer et gérer une base de données locale (`contacts.db`). Une connexion est établie via `_etablir_conn()`, avec `row_factory = sqlite3.Row` pour récupérer les résultats sous forme de dictionnaires.

La table contacts est créée automatiquement si elle n'existe pas, avec les colonnes suivantes : *id (clé primaire auto-incrémentée)*, *nom (obligatoire)*, *prénom (obligatoire)*, *téléphone*, *email*, *adresse*.

4.2.2 Fonctions CRUD

Le modèle implémente les cinq opérations fondamentales :

ajouter_contact() : Insère un nouveau contact avec validation des champs obligatoires.

rechercher_contact() : Recherche avec correspondance partielle (LIKE) sur nom, prénom, téléphone ou email.

modifier_contact() : Met à jour un contact existant identifié par son id.

supprimer_contact() : Supprime un contact par son id.

afficher_tous() : Retourne tous les contacts triés par nom puis prénom (insensible à la casse via COLLATE NOCASE).

Le tri Alphabetique :

Le tri est effectué côté base de données pour garantir des performances optimales :

ORDER BY nom COLLATE NOCASE ASC, prenom COLLATE NOCASE ASC

4.3 Implémentation de la vue (view.py et view2.py)

4.3.1 Vue Tkinter classique (view.py)

Cette version utilise les widgets standards de Tkinter/ttk :

Formulaire : Entry pour les champs simples, Text pour l'adresse.

Liste des contacts : Treeview avec barre de défilement intégrée.

Boutons d'action : Chaque bouton est associé à une icône pour une meilleure lisibilité.

Barre de menu : Navigation structurée (Fichier, Édition, Affichage, Aide).

Barre de statut : Affiche le nombre de contacts visibles.

4.3.2 Vue améliorée avec ttkbootstrap (view2.py)

Grâce à la séparation des responsabilités imposée par l'architecture MVC, il a été possible de développer une seconde vue améliorée avec ttkbootstrap sans modifier la logique métier. Cette version offre une expérience utilisateur modernisée grâce à des thèmes personnalisables, des boutons stylisés et un sélecteur de thème intégré, tout en restant totalement interchangeable avec la vue Tkinter standard

Thèmes personnalisables : L'utilisateur peut changer de thème (clair, sombre, etc.) via un combobox.

Boutons stylisés : Couleurs cohérentes selon la fonction (succès, danger, info, etc.).

Compatibilité ascendante : les mêmes méthodes publiques que `view.py`, garantissant l'interchangeabilité.

4.3.3 Méthodes clés de la vue

`get_inputs()` : Récupère les données saisies dans le formulaire.

`set_inputs(contact)` : Remplit automatiquement le formulaire avec un contact sélectionné.

`afficher_liste(liste_contacts)` : Affiche une liste de contacts dans le Treeview.

`mettre_a_jour_total_contacts()` : Met à jour le compteur dans la barre de statut.

4.4 Implémentation du contrôleur (`controller.py`)

4.4.1 Rôle et responsabilités

Le contrôleur fait le lien entre la vue et le modèle. Il :

- Récupère les saisies utilisateur via `view.get_inputs()`.
- Valide ces données avant toute opération.
- Appelle les méthodes appropriées du modèle.
- Mets à jour la vue après chaque action.

4.4.2 Validation des données

Une validation rigoureuse est effectuée avant chaque écriture en base :

Champs obligatoires : Nom et prénom ne peuvent pas être vides.

Format d'email : vérifié via une expression régulière `(^[^@\\s]+@[^@\\s]+\\.([^@\\s]+)$)`.

Format de téléphone : Accepte les formats locaux et internationaux via une regex adaptée.

Messages d'erreur contextuels : Guidage immédiat en cas de saisie incorrecte.

Note : Les expressions régulières utilisées pour la validation (email, téléphone) ont été élaborées et testées à l'aide de regex101.com, un outil en ligne spécialisé qui permet de construire, déboguer et valider des motifs regex de manière interactive.

4.4.3 Gestion des événements et raccourcis

Le contrôleur lie les boutons et les événements clavier aux actions correspondantes :

Raccourcis clavier : Ctrl+N (ajouter), Ctrl+F (rechercher), Ctrl+M (modifier), Delete (supprimer), etc.

Sélection dans la liste : Un simple clic sur un contact remplit automatiquement le formulaire.

Menu contextuel : Toutes les actions sont accessibles via la barre de menu.

4.5 Point d'entrée (`main.py`)

Le fichier `main.py` initialise les trois composants MVC et gère le fallback automatique :

Si ttkbootstrap est installé, utilise view2.py (interface modernisée).

Sinon, utilise view.py (interface Tkinter standard) avec un message informatif.

Extrait du code (main.py) :

try:

```
import ttkbootstrap
```

```
from view2 import CarnetAdressesView
```

```
except ImportError:from view import CarnetAdressesVie
```

Cette approche garantit la portabilité de l'application même sans dépendances externes.

4.6 Tests unitaires

L'architecture **MVC**, par sa séparation claire des responsabilités, a grandement facilité la mise en place de tests unitaires ciblés. Ainsi, trois suites de tests distinctes ont été développées pour valider chaque composant de manière isolée

4.6.1 Tests du modèle (test_model.py)

Vérifie l'ajout, la recherche, la modification et la suppression de contacts.

Utilise un jeu de données réaliste (26 contacts algériens).

Confirme le tri alphabétique correct.

4.6.2 Tests de la vue (test_view.py)

Teste la récupération et le remplissage des champs du formulaire.

Vérifie le fonctionnement des méthodes `get_inputs()` et `set_inputs()`.

4.6.3 Tests du contrôleur (test_controller.py)

Utilise des objets mock pour isoler les tests.

Vérifie la validation des emails et le bon enchaînement des appels.

4.7 Respect des bonnes pratiques

4.7.1 Séparation des préoccupations

Le modèle ne contient aucune logique d'interface.

La vue ne contient aucune requête SQL.

Le contrôleur ne gère ni l'affichage ni l'accès direct aux données.

4.7.2 Gestion des erreurs

Validation proactive avec des messages utilisateur clairs.

Gestion des exceptions SQLite (transactions via `with`).

Fallback gracieux en cas de dépendance manquante (ttkbootstrap).

4.7.3 Expérience utilisateur

Interface responsive avec redimensionnement.

Feedback visuel immédiat après chaque action.

Raccourcis clavier pour les utilisateurs avancés.

Barre de statut informative.

4.7.4 Le module **logging** a été brièvement utilisé pour le débogage, permettant d'afficher les étapes d'exécution et de monitorer les erreurs directement dans le terminal.

4.7.5 Le module **typing** a été utilisé pour ajouter des annotations de type (type hints), améliorant ainsi la lisibilité du code.

5 Déploiement

Le projet sera entièrement open source et publié sur GitHub avec tous les fichiers nécessaires, garantissant une transparence et une reproductibilité totale.

URL du dépôt : <https://github.com/AnisTEZKRATT/carnet-adresses>

Procédure d'exécution

Prérequis du système : installation de Python, installation de Git (facultative)

Lorsque Git est installé, le dépôt peut être cloné directement.

Dans le cas contraire, le code source peut être téléchargé manuellement depuis le dépôt.

Cloner le dépôt

```
git clone https://github.com/AnisTEZKRATT/carnet-adresses.git
```

```
cd carnet-adresses
```

Installer les dépendances (ttkbootstrap)

```
pip install -r requirements.txt
```

```
python main.py
```

Note : Pour les utilisateurs non techniques, une version exécutable autonome (.exe) peut être générée avec **PyInstaller**, éliminant ainsi la nécessité d'installer Python. Un installateur professionnel

(.msi avec **WiX** ou .exe avec **Inno Setup**) permettrait une installation en quelques clics, une désinstallation propre via le Panneau de configuration.

6. Maintenance

L'architecture MVC sépare clairement les responsabilités. Chaque composant peut être modifié indépendamment, ce qui simplifie la maintenance. Si un problème survient dans une partie, il n'affecte pas forcément les autres.

6.1 Avantages pratiques

Corrections ciblées : Si un bug apparaît dans la base de données (modèle), on le corrige seulement dans `model.py` sans toucher à l'interface.

Évolution simple : Pour ajouter une nouvelle fonctionnalité, on peut souvent modifier un seul composant sans tout réécrire.

6.2 Exemple concret : Ajout de l'export CSV

Cette fonctionnalité, non prévue au départ, a été facile à ajouter grâce à l'architecture MVC :

Dans le contrôleur (`controller.py`) : On a créé une nouvelle méthode `exporter_tout_vers_csv()` qui orchestre l'export.

Dans la vue (`view.py`) : On a juste ajouté un bouton dans le menu "Fichier" pour déclencher cette fonction.

Dans le modèle (`model.py`) : On n'a rien changé car on réutilise la méthode `afficher_tous()` qui existait déjà.

Résultat : L'export CSV fonctionne sans avoir modifié la logique métier ni la structure des données.

6.3 Évolutions futures possibles

Grâce à cette séparation, l'application peut facilement évoluer :

Changer l'interface : On pourrait créer une version web (avec Flask ou Django) en gardant le même modèle et le même contrôleur. L'utilisateur aurait une nouvelle interface, mais ses données resteraient intactes.

Changer de base de données : Si on voulait passer de SQLite à PostgreSQL ou MySQL, on modifierait seulement `model.py`. L'interface utilisateur et la logique métier resteraient les mêmes.

Conclusion

Ce projet de carnet d'adresses a permis de mettre en œuvre concrètement une application bureautique complète en suivant une méthodologie rigoureuse et des principes d'architecture éprouvés.

L'application finalisée répond intégralement aux exigences fonctionnelles initiales, offrant toutes les opérations CRUD via une interface graphique intuitive. L'implémentation de l'architecture MVC a démontré ses avantages tout au long du développement : séparation claire des responsabilités, maintenabilité facilitée et évolutivité garantie.

Les choix technologiques - *Python*, *Tkinter*, *SQLite* - se sont révélés pertinents pour ce type d'application, permettant un développement efficace tout en conservant la simplicité d'usage. L'ajout optionnel de *ttkbootstrap* illustre la flexibilité du système, permettant d'améliorer l'expérience utilisateur sans remettre en cause l'architecture sous-jacente.

L'approche par méthodologie Waterfall, adaptée aux spécifications stables de ce projet, a permis une progression structurée et documentée à chaque étape : analyse, conception, développement et tests.

Enfin, ce projet constitue ainsi une base solide et extensible qui pourrait être enrichie de nombreuses fonctionnalités supplémentaires tout en conservant sa structure propre et maintenable.

Références:

<https://sites.google.com/view/soualah-dz/cours-ihm>

<https://docs.python.org/3>

<https://ttkbootstrap.readthedocs.io>

<https://docs.python.org/3/library/tkinter.html>

<https://plantuml.com>

<https://www.drawio.com>

<https://help.figma.com>

<https://sqlite.org/docs.html>